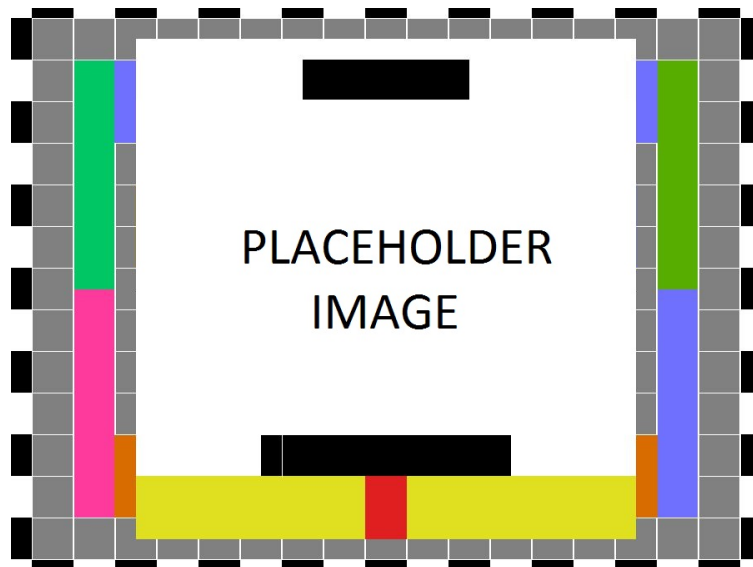


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SPRING 2026**



**TEAM NAME
PRODUCT NAME**

ALAN TURING
GRACE HOPPER
JOHN VON NEUMANN
ADA LOVELACE
CHARLES BABBAGE

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	1.01.2026	GH	document creation
0.2	1.05.2026	AT, GH	complete draft
0.3	1.12.2026	AT, GH	release candidate 1
1.0	1.20.2026	AT, GH, CB	official release
1.1	1.31.2026	AL	added design review requests

CONTENTS

1	Introduction	5
2	Design Stakeholders & Concerns	6
3	Design Viewpoints	7
4	System Overview	8
5	X Layer Subsystems	9
5.1	Layer Hardware	9
5.2	Layer Operating System	9
5.3	Layer Software Dependencies	9
5.4	Subsystem 1	9
5.5	Subsystem 2	10
6	Y Layer Subsystems	11
6.1	Layer Hardware	11
6.2	Layer Operating System	11
6.3	Layer Software Dependencies	11
6.4	Subsystem 1	11
6.5	Subsystem 2	12
7	Z Layer Subsystems	13
7.1	Layer Hardware	13
7.2	Layer Operating System	13
7.3	Layer Software Dependencies	13
7.4	Subsystem 1	13
7.5	Subsystem 2	14
8	Requirements Traceability	15
9	Appendix A	16

LIST OF FIGURES

1	System architecture (carried forward from the ADS)	8
2	Example subsystem description diagram	10
3	Example subsystem description diagram	12
4	Example subsystem description diagram	14

LIST OF TABLES

2	Design stakeholders and concerns	6
3	Design viewpoints (IEEE 1016-2009) used in this document	7
4	Subsystem 1 interfaces	10
5	Subsystem 1 interfaces	12
6	Subsystem 1 interfaces	14
7	Requirement-to-component traceability (excerpt)	15

1 INTRODUCTION

This Detailed Design Specification (DDS) describes the implementation-level design of Product Name. It is organized as a Software Design Description (SDD) in accordance with IEEE Std 1016-2009 [1]: the design is presented as a set of design *views*, each conforming to a design *viewpoint* that frames specific design concerns.

Provide a brief overview of the product concept and a reference to the System Requirements Specification (SRS) and Architectural Design Specification (ADS) in one or two paragraphs. The purpose is to give the reader the location of the background material — the driving requirements and the architecture — that led to the detailed design presented here. The X/Y/Z layer structure introduced in the ADS is carried forward unchanged; this document adds the hardware, software, data, and algorithmic detail needed to build each subsystem.

2 DESIGN STAKEHOLDERS & CONCERNS

IEEE Std 1016-2009 [1] frames a software design description around *design stakeholders* and their *design concerns*. Identify the parties who will use this document — implementers, integrators, testers, and maintainers — and the concerns each holds about the design (for example: how components are decomposed and depend on one another, how data is structured and persisted, how interfaces are contracted, how the system behaves over time, and what hardware/software resources are required). The design viewpoints declared in Section 3 are selected specifically to address these concerns.

Table 2: Design stakeholders and concerns

Design stakeholder	Design concerns
Implementers	composition, interfaces, data structures, algorithms, dependencies
Integrators	interfaces, dependencies, resource/deployment requirements
Testers	interaction and state behavior, interfaces, verifiability
Maintainers	modularity, dependencies, rationale, resource footprint

3 DESIGN VIEWPOINTS

This section declares the design viewpoints (IEEE Std 1016-2009 [1]) used to describe each subsystem in Sections 4–7. A design viewpoint fixes the conventions — the concerns addressed and the notations used — for one kind of design view. Not every subsystem requires every viewpoint; apply those that are relevant and omit the rest (for example, a pure-software module has no hardware resource view). The per-subsystem headings in the layer sections map onto these viewpoints.

Table 3: Design viewpoints (IEEE 1016-2009) used in this document

Design viewpoint	Design concern it addresses	Where described
Context	subsystem boundary and its collaborators	subsystem intro
Composition	how the subsystem is built from components/modules	subsystem intro
Information	data structures, records, schemas, packet formats	Data Structures
Interface	the services/APIs/signals offered and required	Interfaces
Interaction & State	control flow, sequencing, and state dynamics	Data Processing
Algorithm	the procedures/algorithms performed	Data Processing
Resource	hardware, OS, dependencies, languages required	Hardware / OS / Dependencies / Languages

4 SYSTEM OVERVIEW

Reintroduce the full data-flow / architecture diagram from the Architectural Design Specification and briefly (1–2 paragraphs) recap the purpose of each layer, so this document is self-contained. Do not repeat the full architectural analysis; the detail below focuses on *how* each subsystem is implemented. A subsection per layer is not required here.

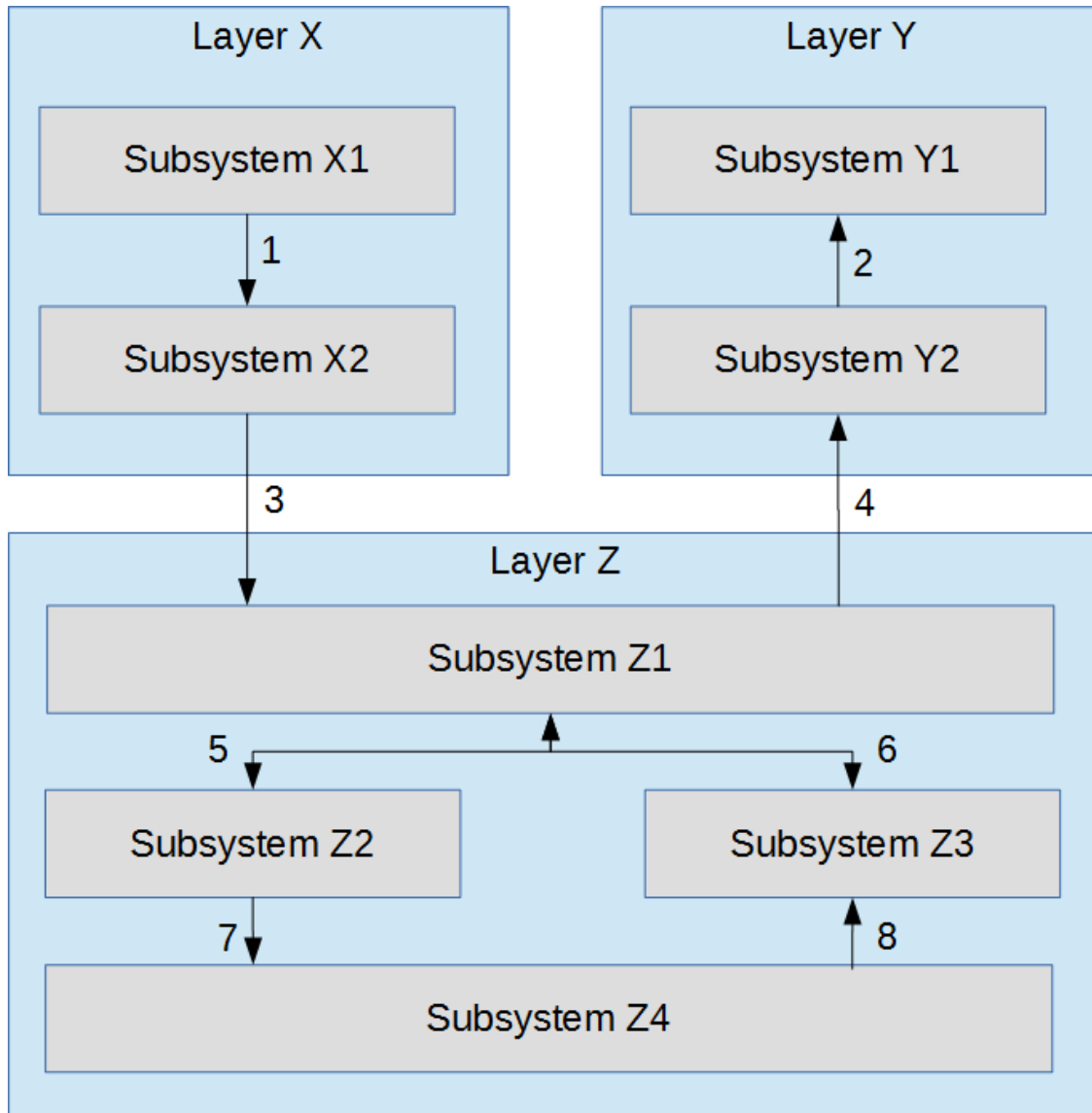


Figure 1: System architecture (carried forward from the ADS)

5 X LAYER SUBSYSTEMS

This section describes the X layer in terms of its hardware and software design. The layer-level subsections below cover properties common to the whole layer (its Resource viewpoint); each subsystem is then described through the design viewpoints declared in Section 3. Include implementation specifics — hardware components, programming languages, software dependencies, operating systems, data structures, and algorithms. Omit any viewpoint that does not apply (for example, a pure-software module has no hardware subsection). The organization and titles below may be adapted as necessary for the project.

5.1 LAYER HARDWARE

Any hardware components common to the layer. For example, if each subsystem is a software process running on the same embedded computer, describe that device here. Do not list hardware that exists only at the subsystem level (cover it in the subsystem sections).

5.2 LAYER OPERATING SYSTEM

Any operating system(s) required by the layer.

5.3 LAYER SOFTWARE DEPENDENCIES

Any software dependencies (libraries, frameworks, runtimes) common to the layer.

5.4 SUBSYSTEM 1

Describe, at a high level, the purpose and basic design of this subsystem (its Context and Composition views): is it a piece of hardware, a class, a process, a web service, or something else, and what components make it up? The items below are specific to this subsystem, not a repeat of the layer-level material above.

5.4.1 SUBSYSTEM HARDWARE

(Resource viewpoint) Any hardware components specific to this subsystem.

5.4.2 SUBSYSTEM OPERATING SYSTEM

(Resource viewpoint) Any operating system specific to this subsystem.

5.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

(Resource viewpoint) Any libraries, frameworks, or design tools specific to this subsystem.

5.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

(Resource viewpoint) Any programming languages used by this subsystem.

5.4.5 SUBSYSTEM DATA STRUCTURES

(Information viewpoint) Any classes, records, schemas, or packet formats worth discussing. For example, data transmitted from a microcontroller to a PC over USB must be assembled into packets — what is the packet structure?

5.4.6 SUBSYSTEM INTERFACES

(Interface viewpoint) The services, APIs, or signals this subsystem offers and requires. Use the interface identifiers established in the ADS so interfaces trace consistently across documents.

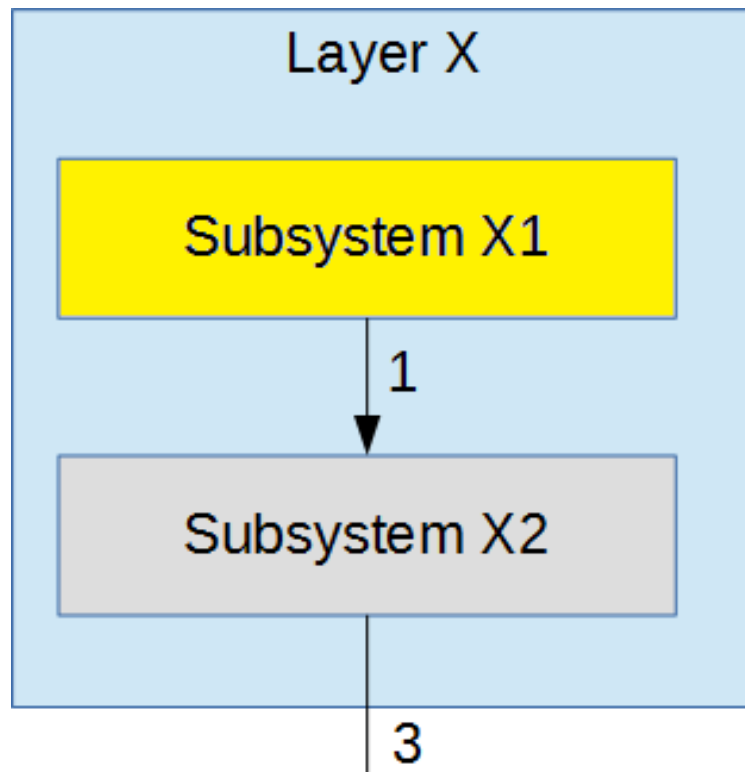


Figure 2: Example subsystem description diagram

Table 4: Subsystem 1 interfaces

ID	Description	Inputs	Outputs
#01	Description of the interface/API/bus	input 1	output 1

5.4.7 SUBSYSTEM DATA PROCESSING

(Algorithm, Interaction & State viewpoints) The algorithms, control flow, and state behavior of the subsystem. If you implement a well-known algorithm, name it; if it is unique to this project, describe it in detail. Where behavior is stateful or sequenced, include a state or sequence diagram.

5.4.8 DESIGN RATIONALE

Justify the key implementation choices for this subsystem (language, library, data structure, algorithm) against the alternatives, and reference the driving requirement IDs and any ADS architecture decisions.

5.5 SUBSYSTEM 2

Repeat for each subsystem in the layer.

6 Y LAYER SUBSYSTEMS

This section describes the Y layer in terms of its hardware and software design. The layer-level subsections below cover properties common to the whole layer (its Resource viewpoint); each subsystem is then described through the design viewpoints declared in Section 3. Include implementation specifics — hardware components, programming languages, software dependencies, operating systems, data structures, and algorithms. Omit any viewpoint that does not apply (for example, a pure-software module has no hardware subsection). The organization and titles below may be adapted as necessary for the project.

6.1 LAYER HARDWARE

Any hardware components common to the layer. For example, if each subsystem is a software process running on the same embedded computer, describe that device here. Do not list hardware that exists only at the subsystem level (cover it in the subsystem sections).

6.2 LAYER OPERATING SYSTEM

Any operating system(s) required by the layer.

6.3 LAYER SOFTWARE DEPENDENCIES

Any software dependencies (libraries, frameworks, runtimes) common to the layer.

6.4 SUBSYSTEM 1

Describe, at a high level, the purpose and basic design of this subsystem (its Context and Composition views): is it a piece of hardware, a class, a process, a web service, or something else, and what components make it up? The items below are specific to this subsystem, not a repeat of the layer-level material above.

6.4.1 SUBSYSTEM HARDWARE

(Resource viewpoint) Any hardware components specific to this subsystem.

6.4.2 SUBSYSTEM OPERATING SYSTEM

(Resource viewpoint) Any operating system specific to this subsystem.

6.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

(Resource viewpoint) Any libraries, frameworks, or design tools specific to this subsystem.

6.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

(Resource viewpoint) Any programming languages used by this subsystem.

6.4.5 SUBSYSTEM DATA STRUCTURES

(Information viewpoint) Any classes, records, schemas, or packet formats worth discussing. For example, data transmitted from a microcontroller to a PC over USB must be assembled into packets — what is the packet structure?

6.4.6 SUBSYSTEM INTERFACES

(Interface viewpoint) The services, APIs, or signals this subsystem offers and requires. Use the interface identifiers established in the ADS so interfaces trace consistently across documents.

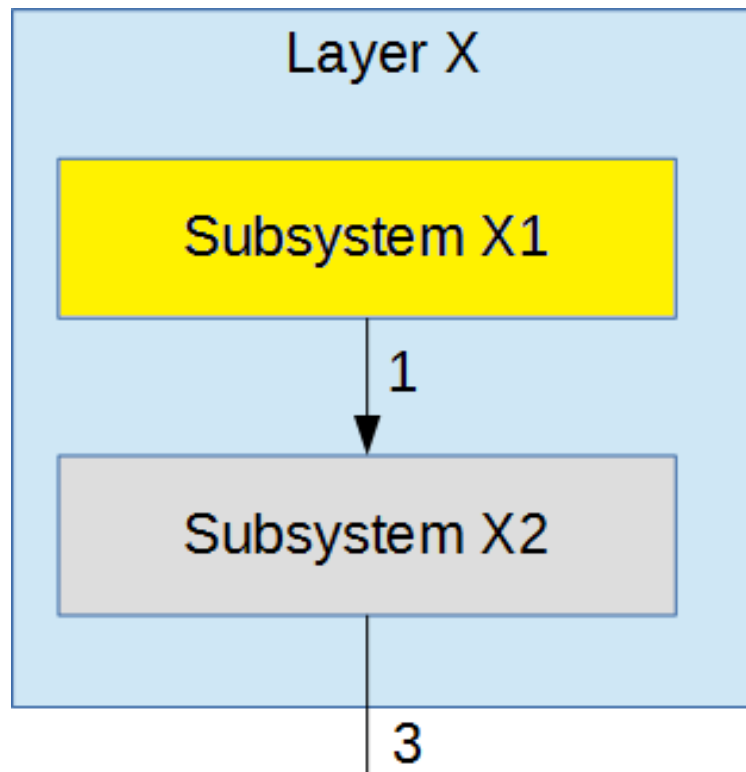


Figure 3: Example subsystem description diagram

Table 5: Subsystem 1 interfaces

ID	Description	Inputs	Outputs
#01	Description of the interface/API/bus	input 1	output 1

6.4.7 SUBSYSTEM DATA PROCESSING

(Algorithm, Interaction & State viewpoints) The algorithms, control flow, and state behavior of the subsystem. If you implement a well-known algorithm, name it; if it is unique to this project, describe it in detail. Where behavior is stateful or sequenced, include a state or sequence diagram.

6.4.8 DESIGN RATIONALE

Justify the key implementation choices for this subsystem (language, library, data structure, algorithm) against the alternatives, and reference the driving requirement IDs and any ADS architecture decisions.

6.5 SUBSYSTEM 2

Repeat for each subsystem in the layer.

7 Z LAYER SUBSYSTEMS

This section describes the Z layer in terms of its hardware and software design. The layer-level subsections below cover properties common to the whole layer (its Resource viewpoint); each subsystem is then described through the design viewpoints declared in Section 3. Include implementation specifics — hardware components, programming languages, software dependencies, operating systems, data structures, and algorithms. Omit any viewpoint that does not apply (for example, a pure-software module has no hardware subsection). The organization and titles below may be adapted as necessary for the project.

7.1 LAYER HARDWARE

Any hardware components common to the layer. For example, if each subsystem is a software process running on the same embedded computer, describe that device here. Do not list hardware that exists only at the subsystem level (cover it in the subsystem sections).

7.2 LAYER OPERATING SYSTEM

Any operating system(s) required by the layer.

7.3 LAYER SOFTWARE DEPENDENCIES

Any software dependencies (libraries, frameworks, runtimes) common to the layer.

7.4 SUBSYSTEM 1

Describe, at a high level, the purpose and basic design of this subsystem (its Context and Composition views): is it a piece of hardware, a class, a process, a web service, or something else, and what components make it up? The items below are specific to this subsystem, not a repeat of the layer-level material above.

7.4.1 SUBSYSTEM HARDWARE

(Resource viewpoint) Any hardware components specific to this subsystem.

7.4.2 SUBSYSTEM OPERATING SYSTEM

(Resource viewpoint) Any operating system specific to this subsystem.

7.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

(Resource viewpoint) Any libraries, frameworks, or design tools specific to this subsystem.

7.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

(Resource viewpoint) Any programming languages used by this subsystem.

7.4.5 SUBSYSTEM DATA STRUCTURES

(Information viewpoint) Any classes, records, schemas, or packet formats worth discussing. For example, data transmitted from a microcontroller to a PC over USB must be assembled into packets — what is the packet structure?

7.4.6 SUBSYSTEM INTERFACES

(Interface viewpoint) The services, APIs, or signals this subsystem offers and requires. Use the interface identifiers established in the ADS so interfaces trace consistently across documents.

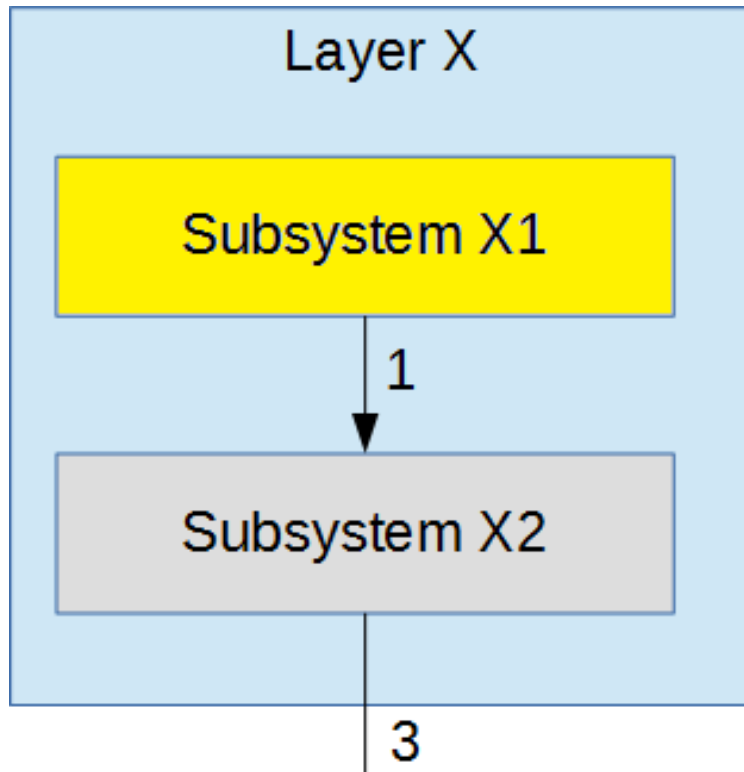


Figure 4: Example subsystem description diagram

Table 6: Subsystem 1 interfaces

ID	Description	Inputs	Outputs
#01	Description of the interface/API/bus	input 1	output 1

7.4.7 SUBSYSTEM DATA PROCESSING

(Algorithm, Interaction & State viewpoints) The algorithms, control flow, and state behavior of the subsystem. If you implement a well-known algorithm, name it; if it is unique to this project, describe it in detail. Where behavior is stateful or sequenced, include a state or sequence diagram.

7.4.8 DESIGN RATIONALE

Justify the key implementation choices for this subsystem (language, library, data structure, algorithm) against the alternatives, and reference the driving requirement IDs and any ADS architecture decisions.

7.5 SUBSYSTEM 2

Repeat for each subsystem in the layer.

8 REQUIREMENTS TRACEABILITY

Extend the traceability record from the SRS and ADS down to the component level. Every requirement should trace to the specific subsystem/component that implements it and to the interface(s) through which it is realized; this is the coverage evidence carried into the Test Plan. Mark any requirement not yet designed as “TBD” and resolve it before design freeze.

Table 7: Requirement-to-component traceability (excerpt)

Req ID	Requirement (abbreviated)	Component	Interface
FR-01	abbreviated requirement text	X.Subsystem 1	#01
PE-01	abbreviated requirement text	Y.Subsystem 2	#03

9 APPENDIX A

Include any additional documents (CAD design, circuit schematics, etc) as an appendix as necessary.

REFERENCES

- [1] IEEE, “IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions,” Institute of Electrical and Electronics Engineers, New York, NY, USA, Tech. Rep., 2009.